

Informe Final del Proyecto

Clavados en Ciudad Academia

Integrantes:

Camilo Andres Villanueva Gonzalez

Jose Manuel Posada Londoño

Curso: Informática II / Programación Orientada a Objetos

Universidad de Antioquia

5 de junio de 2026

Resumen

Clavados en Ciudad Academia es un videojuego 2D desarrollado en C++ con Qt, ambientado en una competencia deportiva de clavados dentro de un universo especial inspirado en Ciudad Academia. El proyecto integra programación orientada a objetos, memoria dinámica, interfaz gráfica, modelos físicos, dificultad configurable, sonido, sprites, un agente inteligente y dos niveles jugables con dinámicas diferenciadas. El objetivo principal fue construir una experiencia completa tanto visual como funcional.

1. Introducción

La propuesta combina clavados, con un universo futurístico en el que los personajes poseen especiales. Se debe controlar la caída del personaje desde plataformas elevadas, atravesar zonas de viento, recolectar monedas metálicas y entrar correctamente en la piscina.

Todo esto mediante programación orientada a objetos, memoria dinámica, manejo de eventos, interfaz gráfica, modelos físicos y diseño modular.

2. Planteamiento del problema

Para este proyecto nos enfocamos en la funcionalidad y la estética del diseño, que no se redujera a mover un sprite en pantalla. El software debía demostrar una arquitectura clara, separación entre lógica y GUI, uso de memoria dinámica, herencia propia, tres modelos físicos no triviales, dificultad configurable y un agente inteligente con percepción, razonamiento, acción y aprendizaje.

Además, el juego debía ser agradable para el usuario final. Por esta razón se trabajó en el HUD, sprites, sonidos, menú inicial, pantalla de pausa, game over, textura de entorno, piscinas, plataformas y retroalimentación visual.

3. Definición general y objetivos

3.1. Idea general

Clavados en Ciudad Academia es un juego de clavados con dos niveles principales:

- **Nivel 1: Piscina de entrenamiento.** El jugador salta desde una plataforma, controla su caída y debe entrar correctamente en una piscina móvil.
- **Nivel 2: Torre experimental.** El jugador salta desde mayor altura, enfrenta viento, ráfagas, obstáculos, monedas metálicas, proyectiles del dron vigilante y una piscina final móvil.

3.2. Objetivo general

Desarrollar un videojuego 2D en C++ con Qt que integre POO, memoria dinámica, GUI, modelos físicos, dificultad configurable, sonido, sprites y un agente inteligente.

3.3. Objetivos específicos

- Implementar una arquitectura orientada a objetos con clases base y clases derivadas.
- Usar memoria dinámica de forma segura mediante `std::unique_ptr`.
- Diseñar dos niveles con dinámicas diferenciadas.

- Aplicar modelos físicos: gravedad parabólica, oscilación, viento/turbulencia, colisiones y movimiento de piscina.
- Incorporar un agente inteligente representado por un dron vigilante.
- Implementar HUD, sonidos, menú, pausa, game over y recursos gráficos.
- Generar un ejecutable funcional.

4. Especificación de requerimientos

4.1. Requisitos funcionales

Requisito	Descripción
Menú inicial	Permite seleccionar dificultad y personaje.
Dos niveles jugables	Piscina de entrenamiento y torre experimental.
Control del personaje	Movimiento lateral, aceleración/desaceleración de caída y poderes especiales.
Personajes seleccionables	Mikoto Misaka, Accelerator, Mugino y Dark Matter.
Poderes físicos	Cada personaje modifica la interacción con monedas o trayectoria.
Sistema de vidas	Se descuentan intentos al fallar la entrada.
Game over	Cuando se acaban las vidas, se muestra pantalla de derrota y sonido.
Sonidos	Música de menú, fondo y sonidos de eventos.
Agente inteligente	Dron que percibe, razona, actúa y aprende.
Ejecutable	Archivo ejecutable generado para entrega.

5. Metodología y planificación

Se empleó una metodología incremental. Primero se definió la idea del juego y los niveles. Luego se modeló la arquitectura lógica, las entidades y el agente inteligente. Finalmente se implementó y se pulió la interfaz, los sprites, sonidos, físicas, dificultad, manejo de errores y ejecutable.

Fase	Actividades
Momento I	Definición de temática, niveles, personajes, físicas y agente.
Momento II	Diseño de clases, relaciones, lógica del agente y sprites.
Momento III	Implementación final, GUI, sonidos, dificultad, ejecutable y documentación.

6. Diseño y arquitectura del sistema

El proyecto está organizado en carpetas que separan responsabilidades:

- entidades/: clases del mundo del juego.
- fisicas/: modelos físicos y ecuaciones auxiliares.
- agente/: dron vigilante inteligente.
- logica/: niveles, dificultad y excepciones.
- gui/: interfaz Qt y estados de pantalla.
- render/: cache de sprites.
- recursos/: sprites, sonidos e imágenes.

6.1. Diagrama resumen de clases

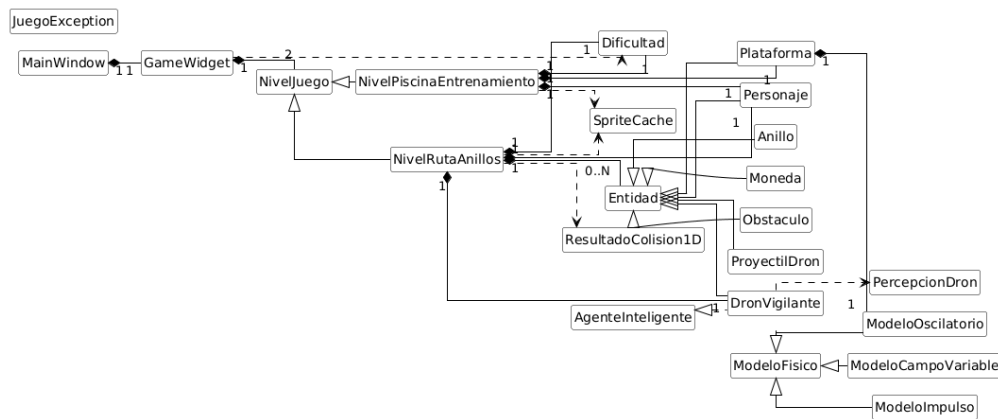


Figura 1: Menú principal del juego.

6.2. Separación entre lógica y GUI

La clase `GameWidget` se encarga de la interfaz Qt, los estados de pantalla, menú, pausa, game over, sonidos y dibujo general. La lógica de cada nivel está separada en `NivelPiscinaEntrenamiento` y `NivelRutaAnillos`. Esto permite que cada nivel tenga reglas propias sin mezclar todo en la interfaz.

7. Programación orientada a objetos y memoria dinámica

7.1. Herencia propia

El proyecto cumple con herencia propia mediante clases que no heredan de Qt:

- Personaje, Moneda, ProyectilDron, Anillo, Obstaculo heredan de Entidad.
- NivelPiscinaEntrenamiento y NivelRutaAnillos heredan de NivelJuego.

La herencia permite reutilizar comportamientos comunes.

Por ejemplo:

```
Entidad
|
|--- Personaje
|
|--- Plataforma
|
|--- Obstaculo
|
|--- Anillo
|
|--- ZonaViento
```

7.2. Abstracción

Cada entidad del videojuego representa una abstracción de un elemento del mundo real o del entorno ficticio.

Por ejemplo:

- Personaje.
- Plataforma.
- Dron.
- Obstáculo.

7.3. Encapsulamiento

Cada clase administra internamente sus atributos y expone únicamente las operaciones necesarias para interactuar con otras clases.

Esto permite reducir el acoplamiento entre componentes.

7.4. Polimorfismo

El polimorfismo permite manejar distintos tipos de entidades mediante referencias a una clase base común.

Esto simplifica considerablemente la gestión de objetos durante la ejecución.

7.5. Memoria dinámica

Se usa memoria dinámica segura mediante `std::unique_ptr`. Por ejemplo, los niveles almacenan entidades así:

```
std::unique_ptr<Personaje> jugador;  
std::unique_ptr<DronVigilante> dron;  
std::vector<std::unique_ptr<Moneda>> monedas;  
std::vector<std::unique_ptr<ProyectilDron>> proyectilesDron;
```

Esto evita fugas de memoria porque los objetos se liberan automáticamente cuando salen de alcance. Además, las clases base poseen destructores virtuales, lo cual permite destruir objetos derivados correctamente.

8. Uso de contenedores STL

Las estructuras STL permiten almacenar y administrar conjuntos de objetos de manera eficiente.

Algunos contenedores útiles en este proyecto son:

- `std::vector` para guardar niveles, entidades y obstáculos.
- `std::map` para almacenar parámetros aprendidos por el agente.
- `std::vector<float>` para datos numéricos recientes.

9. Integración con Qt

Qt fue utilizado como base para la interfaz gráfica del videojuego. Gracias a este framework fue posible implementar:

- Ventanas principales.
- Escenas de juego.
- Manejo de eventos.
- Temporizadores.
- Sprites.
- Sonidos.

Otras clases:

- `QWidget`
- `QGraphicsScene`

- QGraphicsView
- QTimer
- QSoundEffect

La integración de estos elementos permite que el videojuego funcione como una aplicación interactiva completa.

10. Descripción detallada de la capa lógica

La capa lógica del videojuego representa el corazón del sistema, debido a que allí se definen las reglas internas, el comportamiento de los objetos, la transición entre niveles y la interacción entre las distintas entidades.

La decisión de separar esta capa de la interfaz gráfica permite que el proyecto mantenga una estructura más limpia, modular y fácil de mantener. Además, facilita la reutilización de clases y el uso de polimorfismo en diferentes partes del videojuego.

10.1. Clase Juego

La clase **Juego** se encarga de coordinar el funcionamiento general de la aplicación. Su responsabilidad principal consiste en administrar el estado global de la partida, decidir qué nivel se encuentra activo y controlar las transiciones entre pantallas.

Entre sus funciones más importantes se encuentran:

- Inicializar la partida.
- Cargar el nivel correspondiente.
- Supervisar el avance del jugador.
- Determinar si el nivel fue superado o fallado.
- Reiniciar el juego cuando sea necesario.

Esta clase actúa como punto central de coordinación entre la lógica, la interfaz y los recursos multimedia.

10.2. Clase Nivel

La clase **Nivel** representa una abstracción general de cualquier escenario del videojuego. En ella se almacenan los atributos y comportamientos comunes a todos los niveles.

Normalmente, esta clase contiene elementos como:

- Entidades activas.
- Puntaje acumulado.

- Estado del nivel.
- Tiempo restante.
- Condiciones de victoria.

La existencia de esta clase facilita la implementación de herencia, ya que permite definir versiones especializadas para cada escenario.

10.3. NivelPiscinaEntrenamiento

Esta clase representa el primer nivel del juego. Su objetivo es introducir al jugador en las mecánicas básicas.

Desde el punto de vista lógico, este nivel se caracteriza por:

- Tener una cámara fija.
- Presentar una dificultad moderada.
- Utilizar una plataforma oscilante.
- Evaluar principalmente la postura y la entrada al agua.

El nivel de entrenamiento funciona como una etapa inicial en la que el jugador aprende a dominar el salto, el control aéreo y la precisión en la inmersión.

10.4. NivelRutaAnillos

Esta clase implementa el segundo nivel, que corresponde a una versión más compleja y exigente del juego.

En esta etapa el jugador debe enfrentarse a:

- Mayor altura.
- Scroll vertical limitado.
- Obstáculos flotantes.
- Anillos de bonificación.
- Zonas con viento.
- Zonas de velocidad variable.
- Temporizador.
- Intervención del dron supervisor.

Este nivel está pensado para probar el dominio del jugador bajo presión y con múltiples variables que modifican el recorrido del salto.

11. Descripción de las entidades del juego

Las entidades representan todos los objetos que pueden aparecer dentro del mundo del videojuego. Cada una de ellas posee una posición, unas dimensiones, un comportamiento y, en algunos casos, un estado dinámico.

11.1. Clase Entidad

La clase **Entidad** funciona como clase base para cualquier objeto interactivo o visible dentro del juego.

Sus atributos más comunes son:

- Posición en el espacio.
- Tamaño.
- Estado de activación.
- Representación gráfica.

La ventaja de esta clase es que permite tratar varios objetos diferentes de manera uniforme mediante punteros o referencias a la clase base.

11.2. Clase Personaje

La clase **Personaje** representa a Mikoto Misaka, el personaje principal del videojuego.

Esta clase no solo contiene información visual, sino también variables relacionadas con la física del movimiento. Por ejemplo:

- Velocidad horizontal.
- Velocidad vertical.
- Aceleración.
- Masa.
- Energía disponible.
- Estado de salto.
- Estado de caída.
- Estado de impulso electromagnético.

El personaje puede realizar acciones como saltar, corregir su postura en el aire y usar un impulso especial limitado.

11.3. Clase Plataforma

La plataforma es el punto de inicio del clavado. En el primer nivel tiene un comportamiento oscilatorio, lo que introduce dificultad adicional al momento de decidir el instante del salto.

Sus características principales son:

- Posición base.
- Amplitud de oscilación.
- Frecuencia.
- Velocidad de movimiento.

11.4. Clase Obstaculo

Los obstáculos aparecen principalmente en el segundo nivel. Su función es impedir un recorrido limpio y obligar al jugador a reaccionar de forma precisa.

Dependiendo del diseño específico del nivel, un obstáculo puede ser estático o dinámico. Algunos pueden moverse ligeramente para aumentar la dificultad.

11.5. Clase AnilloBonificacion

Los anillos de bonificación representan objetivos opcionales dentro del segundo nivel. Atravesarlos otorga puntos adicionales y premia la precisión del jugador.

Su presencia hace que el recorrido no se limite solamente a llegar al agua, sino también a optimizar la trayectoria de la caída.

11.6. Clase Moneda

La clase `Moneda` representa uno de los elementos recolectables del segundo nivel. A diferencia de los anillos de bonificación, las monedas están pensadas para interactuar directamente con los poderes de los personajes. Esto permite que no sean únicamente objetos estáticos de puntaje, sino elementos que pueden moverse, acercarse, desplazarse o cambiar su trayectoria según la habilidad activa del jugador.

11.7. Clase ZonaViento

La clase `ZonaViento` representa regiones del escenario donde el personaje recibe una fuerza lateral adicional.

Estas zonas son importantes porque alteran la trayectoria de manera local y obligan al jugador a corregir el movimiento durante el descenso.

12. Modelos físicos implementados

12.1. Integración con delta de tiempo

El juego actualiza la física con un intervalo aproximado de:

$$dt = 0,016 \text{ s}$$

Esto representa cerca de 60 cuadros por segundo. En lugar de mover objetos con valores fijos por frame, se usan ecuaciones dependientes del tiempo:

$$v = v + a \cdot dt$$

$$x = x + v_x \cdot dt$$

$$y = y + v_y \cdot dt$$

12.2. Movimiento parabólico

El salto y la caída del personaje se modelan con velocidad inicial y gravedad:

$$v_y = v_{y0} + g \cdot t$$

$$y = y_0 + v_{y0}t + \frac{1}{2}gt^2$$

En el código se implementa mediante integración por pasos:

```
FisicaJuego::integrarVelocidad(vy, ay, dt);  
FisicaJuego::integrarPosicion(y, vy, dt);
```

12.3. Viento y turbulencia

El viento lateral altera la aceleración horizontal:

$$a_x = I + T \sin(\omega t + ky)$$

donde I es la intensidad base y T representa la turbulencia. Esto hace que la trayectoria no sea completamente predecible.

12.4. Movimiento oscilatorio

La piscina y algunos elementos usan movimiento oscilatorio:

$$x = x_{base} + A \sin(\omega t)$$

Esto permite que la piscina se mueva horizontalmente y aumente la dificultad.

12.5. Colisiones elásticas e inelásticas

Para los proyectiles del dron se implementan choques basados en momento lineal. La fórmula general usada es:

$$v'_1 = \frac{(m_1 - em_2)v_1 + (1 + e)m_2v_2}{m_1 + m_2}$$
$$v'_2 = \frac{(m_2 - em_1)v_2 + (1 + e)m_1v_1}{m_1 + m_2}$$

donde e es el coeficiente de restitución. Si $e = 1$, la colisión es elástica. Si $0 < e < 1$, es inelástica.

13. Sistema de dificultad

La dificultad se va a ver afectada debido a estos aspectos:

- intensidad del viento,
- factor de gravedad,
- movimiento de piscina,
- fuerza de ráfagas,
- presión del agente inteligente,
- energía inicial,
- intentos disponibles,
- puntaje mínimo.

Esto permite que el comportamiento del juego cambie de forma real entre fácil, normal y difícil.

14. Agente inteligente

El agente inteligente es el `DronVigilante`. Su diseño sigue cuatro componentes:

14.1. Percepción

El dron observa datos del jugador:

- posición,
- distancia,

- velocidad,
- dirección,
- si está usando impulso,
- predicción de trayectoria.

14.2. Razonamiento

Con base en la percepción, el dron cambia de estado:

- PATRULLA,
- ESCANEO,
- ANTICIPA,
- INTERCEPTA.

14.3. Acción

El dron se mueve, sigue al jugador, calcula disparos predictivos y lanza proyectiles, los proyectiles pueden producir colisiones elásticas o inelásticas.

14.4. Aprendizaje

El dron guarda memoria de:

- errores de entrada a la piscina,
- aciertos del jugador,
- impactos recibidos,
- evasiones de proyectiles.

Con esa información ajusta su presión de dificultad.

15. Desarrollo e implementación

15.1. Personajes

El jugador puede elegir entre cuatro personajes:

- **Mikoto Misaka**: campo electromagnético para atraer monedas.
- **Accelerator**: control vectorial para desplazar monedas.

- **Mugino:** energía Meltdowner con comportamiento lateral.
- **Dark Matter:** manipulación orbital de materia oscura.

Cada personaje tiene sprites propios, masa, energía, control lateral, arrastre y poder físico.

15.2. HUD e interfaz

El HUD muestra:

- puntaje,
- energía,
- monedas recolectadas,
- vidas o golpes,
- estado del poder,
- estado del viento,
- progreso de descenso.

También se implementaron menú inicial, pausa, pantalla de victoria, pantalla de game over y retorno al menú.

15.3. Optimización

Para mejorar eficiencia:

- se usa SpriteCache para reutilizar sprites escalados;
- se evita cargar imágenes dentro del ciclo de dibujo;
- se usan reserve() en vectores;
- se usan unique_ptr para evitar fugas de memoria;
- se evita copiar QPixmap innecesariamente en el dibujo del personaje.

16. Procedimientos de prueba

Prueba	Procedimiento	Resultado esperado
Inicio del juego	Ejecutar el archivo final	-Se abre el menú principal.
Selección de personaje	Elegir cada personaje	-Cambian sprites y poderes.
Nivel 1	Saltar y entrar a la piscina	-Se calcula puntaje y se descuenta vida si falla.
Nivel 2	Saltar desde la torre	-Se activa trayectoria parabólica y aparecen obstáculos.
Game over	Perder todos los intentos	-Aparece pantalla de game over y vuelve al menú.
Sonidos	Probar salto, colisión, agua y menú	-Se reproducen eventos correctamente.
Pantalla completa	Presionar F11	-La interfaz se adapta a la pantalla.

16.1. Controles

Tecla	Acción
Espacio	Iniciar salto.
A / Flecha izquierda	Corrección hacia la izquierda.
D / Flecha derecha	Corrección hacia la derecha.
W / Flecha arriba	Frenar o desacelerar caída.
S / Flecha abajo	Acelerar caída.
Click / E	Activar poder físico.
R	Reiniciar intento o nivel.
Esc	Pausar.
F11	Pantalla completa.

17. Resultados y discusión

El resultado final es un juego con dos niveles, personajes seleccionables, física de caída, viento, movimiento de piscina, proyectiles, sonido, HUD, menú, game over y agente inteligente. El proyecto cumple con todos los elementos propuestos para el desafío.

Entre las dificultades enfrentadas estuvieron la sincronización entre sprites y colisiones, el ajuste de la escala en pantalla completa, la integración de sonidos, la corrección de la cámara vertical y la creación de una dificultad que no dependiera únicamente del tiempo y por ultimo la integracion del agente inteligente.

18. Conclusiones

Este proyecto permitió aplicar todos los conceptos vistos en el curso, la programación orientada a objetos facilitó dividir el problema en: niveles, física, agente, dificultad e interfaz, también el uso de memoria dinámica con punteros permitió manejar objetos del juego sin fugas de memoria, los modelos físicos hicieron que el juego tuviera esencia.

El agente inteligente aportó un componente investigativo importante, ya que percibe al jugador, toma decisiones, dispara proyectiles y aprende de los resultados y también, el desarrollo del HUD, sprites, sonido y ejecutable permitió transformar el videojuego estéticamente bonito.

19. Referencias

- Documentación oficial de Qt: <https://doc.qt.io/>
- Referencia de C++: <https://en.cppreference.com/>
- Repositorio del proyecto: <https://github.com/josemanuelposada-collab/Clavados-en-Ciudad-academia>
- Video de sustentación: <https://youtu.be/He5mfB9LE08?si=gSiVk9n7RLVSN947>
- Trailer del juego: https://youtu.be/4vusR0_JbAU?si=j7YJk2Iwbm_QPbeh

A. Anexo A: Capturas del juego

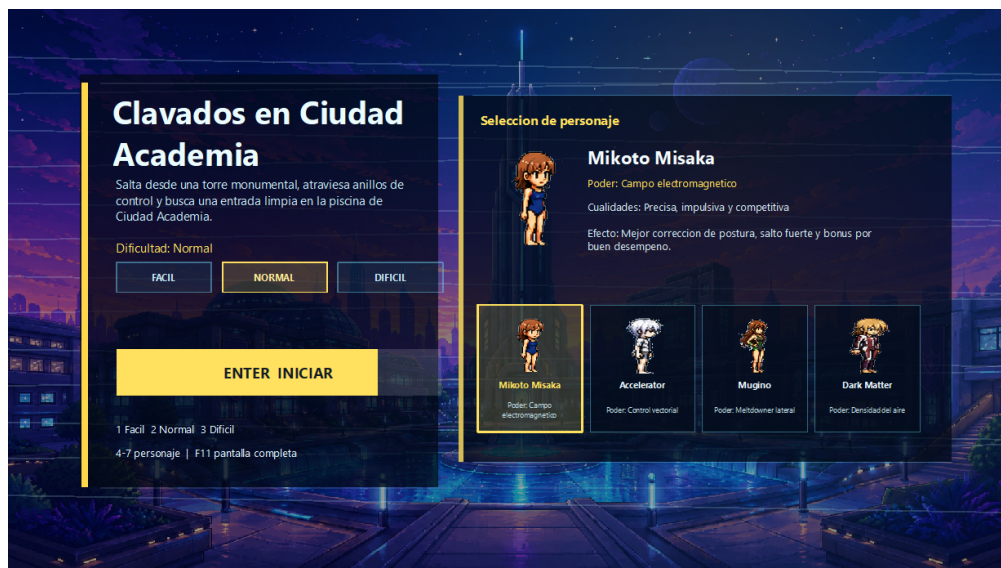


Figura 2: Menú principal del juego.

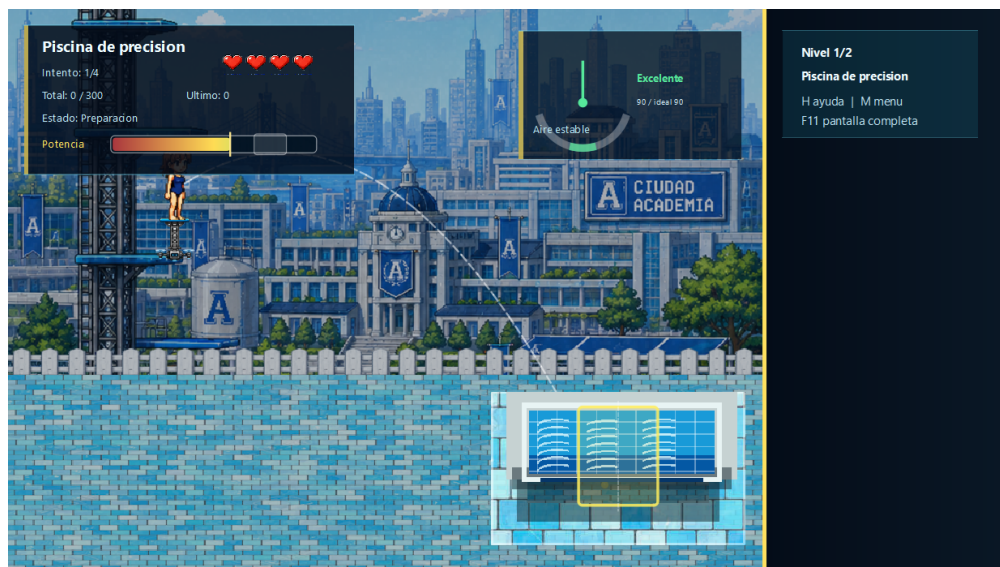


Figura 3: Nivel 1: Piscina de entrenamiento.

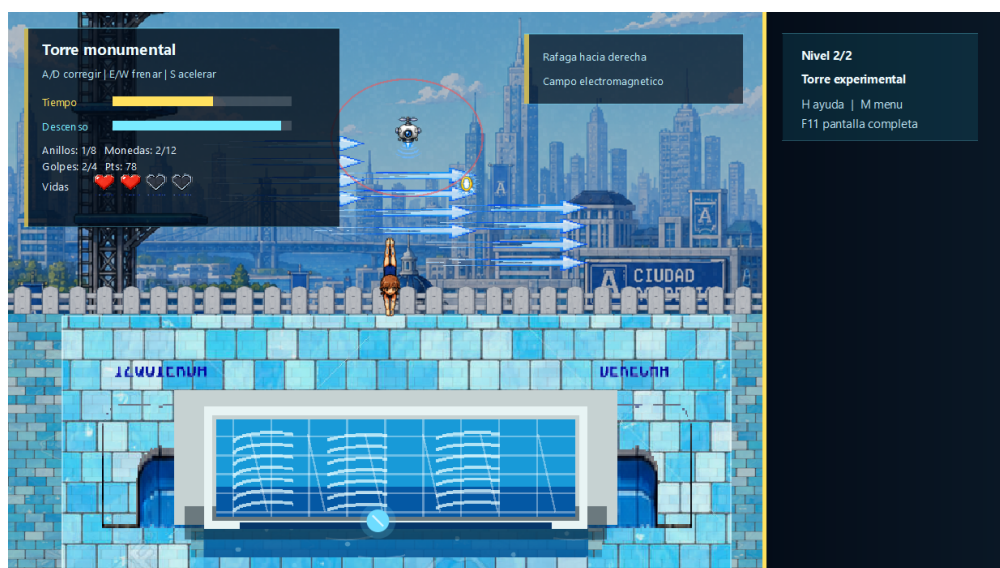


Figura 4: Nivel 2: Nivel ruta de anillos.